

KUBERNETES FOUNDATION

なぜ Kubernetes が 必要なのか

コンテナ単体運用の限界を
宣言的モデルと調整ループで自動的に埋める

コンテナオーケストレーション

宣言的モデル

調整ループ

| Key Takeaway

コンテナ単体では本番運用が回らない。Kubernetes は望ましい状態の宣言と調整ループで、復旧・スケール・更新を自動化する

01

オーケストレーションが要る

手動のコンテナ運用は復旧もスケールも追いつかない。自動化が前提になる

02

宣言的に管理する

あるべき状態を YAML で宣言し、調整ループが現状との差分を埋め続ける

03

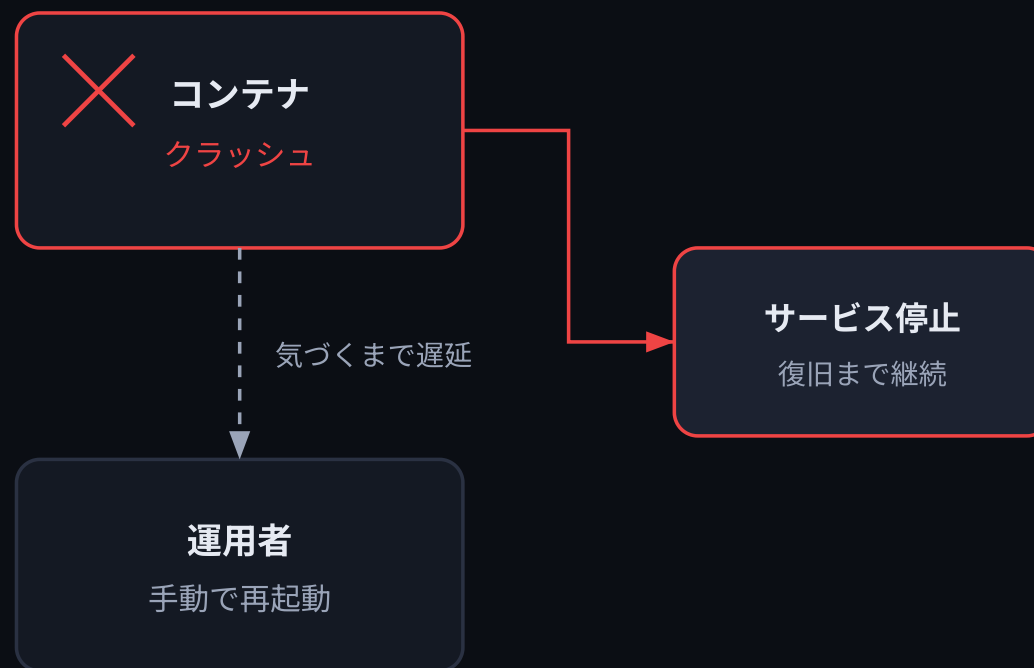
運用が自動で回る

自己修復・水平スケール・ローリング更新で人手の介入を減らせる

| コンテナ単体では本番が回らない

docker run は開発では便利でも、落ちたら誰かが気づいて建て直すまでサービスは止まる。

- クラッシュしても自動復旧しない
- 負荷増は手作業でスケール
- 複数サーバーへの配置が煩雑



I 宣言的モデルが手順を肩代わりする

命令的

起動

確認

再起動

宣言的

desired state
replicas: 3

あとは
Kubernetes が実行

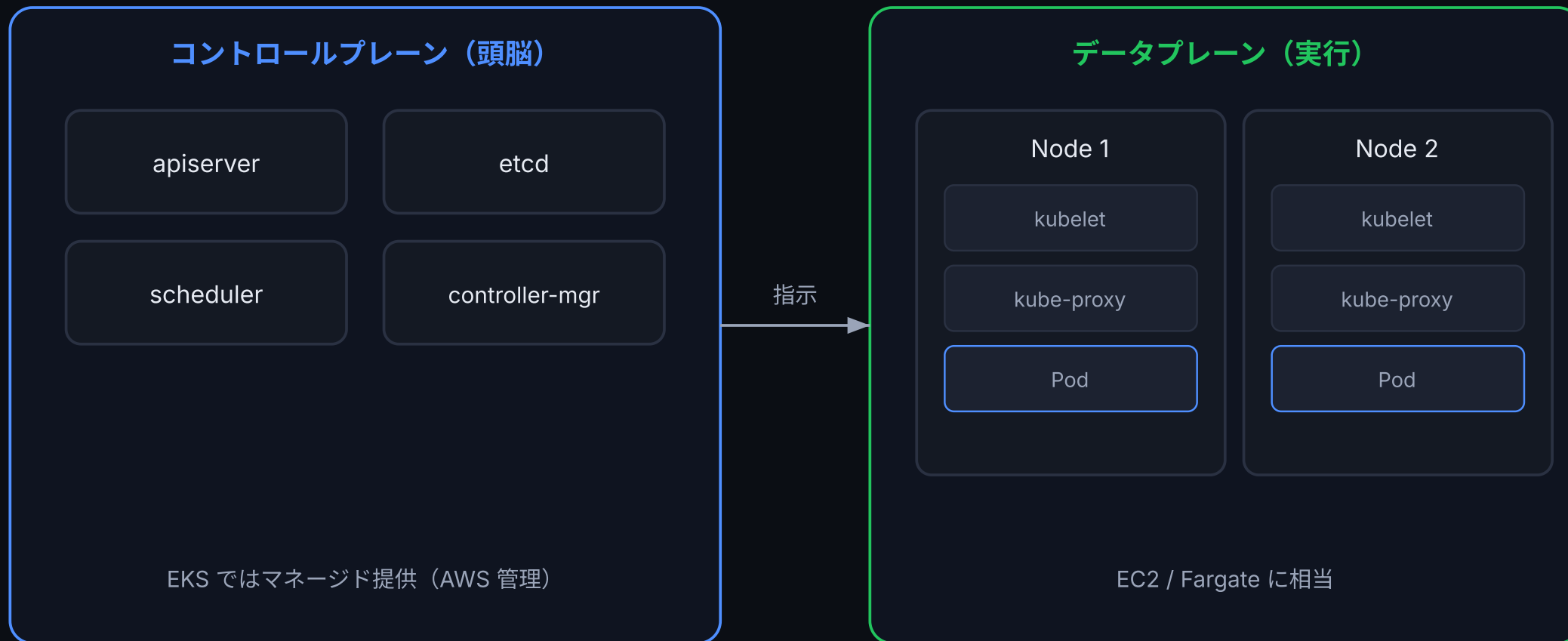
命令的は手順を一つずつ指示する。宣言的は最終状態だけを宣言し、到達方法はシステムに任せる。

- 命令的＝起動・確認・再起動を逐次指示
- 宣言的＝あるべき状態を YAML で宣言
- kubectl apply で適用すれば後は自動

調整ループが差分を埋め続ける



！ クラスタは「頭脳」と「実行」に分かれる



Ⅰ 運用タスクは手動から自動へ

運用タスク	コンテナ単体	Kubernetes
障害復旧	運用者が手動で再起動	自己修復で自動再作成
スケール	手作業でコピー	HPA で自動増減
更新	全停止してから入替	ローリング更新
配置	どこに置くか人が判断	scheduler が自動配置

AWS で言えば ECS Service Auto Scaling や Service Scheduler に相当する自動化を、クラウド非依存で実現する。

I まとめ

01

オーケストレーション必須

コンテナ単体運用は復旧もスケールも限界。
自動化の仕組みが要る

02

宣言的+調整ループ

望ましい状態を宣言すれば、頭脳（CP）と
実行（DP）が差分を埋め続ける

03

ポータビリティ

どのクラウドでも同じスキルが活きる。AWS
知識を踏み台に学べる